

STAT 4360 Statistical Learning, Fall 2024
Mini Project 1
By: Benny Frisella

1.

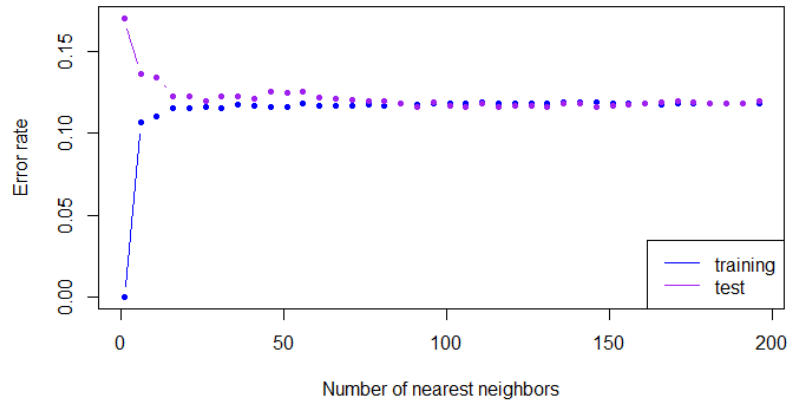


Figure 1

Figure 1 shows a Test MSE vs. Training MSE model consistent with what I've seen in class. The test MSE starts high while the training MSE starts low. They converge quickly. We look for the minimum test error rate and the minimum difference between the two rates.

The optimal value of k was found at $k = 131$, where the test MSE was minimized. Remarkably, the difference between test MSE and training MSE was also minimized at this point.

At $k = 131$ nearest neighbors, the training MSE was 0.1185 while test MSE was 0.116.

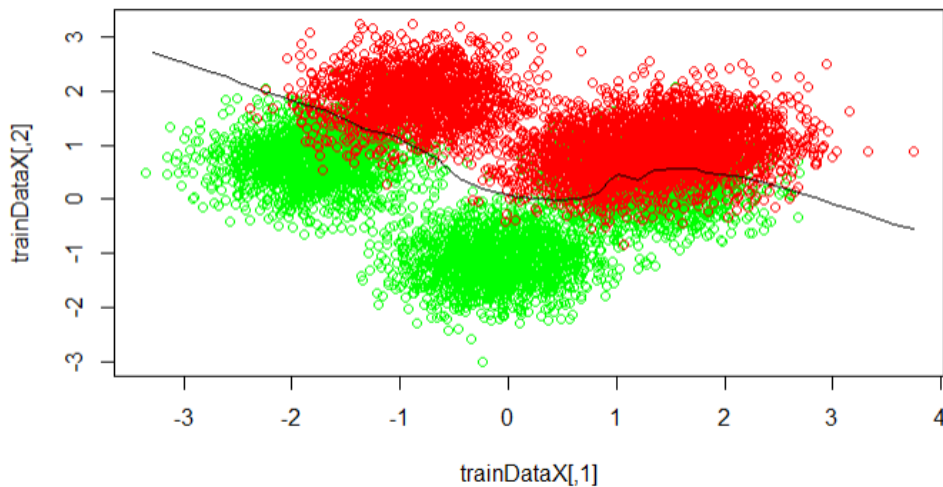


Figure 2

Figure 2 shows the decision boundary. Along the boundary through the negative x-axis the densities appear sparser as the boundary does a good job of separating the two classes. However, while it's still decent through the positive x-axis, I think there is room for improvement there. Overall, I believe the boundary looks good, which means our model should be quite accurate. I don't know enough tests to dive deeper into it at this moment.

2. Take $Y_i = \hat{f}(x_i) + \epsilon$ as a general model, then
allow $Y_0 = \hat{f}(x_0) + \epsilon$ as a test model where ϵ are independent, identically distributed.

$$\text{Let } \hat{f}(x) = \hat{\beta}$$

Showing $\text{MSE}\{\hat{\beta}\} = \{\text{Bias}(\hat{\beta})\}^2 + \text{Var}(\hat{\beta})$

$$\begin{aligned} E[\hat{\beta} - \beta]^2 &= [E[\hat{\beta}] - \beta]^2 + E[\hat{\beta} - E[\hat{\beta}]]^2 \\ &= E[\hat{\beta} - E[\hat{\beta}] + E[\hat{\beta}] - \beta]^2 \\ &= E[(\hat{\beta} - E[\hat{\beta}])^2 + (E[\hat{\beta}] - \beta)^2 + 2 * (\hat{\beta} - E[\hat{\beta}])(E[\hat{\beta}] - \beta)] \\ &= E[(\hat{\beta} - E[\hat{\beta}])^2] + E[(E[\hat{\beta}] - \beta)^2] + 2 * E[(\hat{\beta} - E[\hat{\beta}])(E[\hat{\beta}] - \beta)] \\ &= \text{variance} + \text{bias}^2 + 0 \end{aligned}$$

$$E[(\hat{Y}_0 - Y_0)]^2 \quad \text{let } \hat{Y}_0 = \hat{f}(x_0)$$

$$\begin{aligned} &= E[(\hat{f}(x_0) - E[\hat{f}(x_0)]) + (E[\hat{f}(x_0)] - Y_0)]^2 \quad (\text{add and subtract same term}) \\ &= E[(\hat{f}(x_0) - E[\hat{f}(x_0)]) + (E[\hat{f}(x_0)] - Y_0)]^2 \quad (\text{Rearrange terms}) \\ &= E[(\hat{f}(x_0) - E[\hat{f}(x_0)])^2 + 2(\hat{f}(x_0) - E[\hat{f}(x_0)])(E[\hat{f}(x_0)] - Y_0) + (E[\hat{f}(x_0)] - Y_0)^2] \\ &\quad (\text{Using the binomial formula: } (a + b)^2 = a^2 + 2ab + b^2) \\ &= E[(\hat{f}(x_0) - E[\hat{f}(x_0)])^2] + 2E[(\hat{f}(x_0) - E[\hat{f}(x_0)])(E[\hat{f}(x_0)] - Y_0)] + E[(E[\hat{f}(x_0)] - Y_0)^2] \\ &= \text{Var}[\hat{f}(x_0)] + 0 + (\text{Bias}[\hat{f}(x_0)] + (f(x_0) - Y_0))^2 \\ &= \text{Var}[\hat{f}(x_0)] + (\text{Bias}[\hat{f}(x_0)])^2 + 2 \text{Bias}[\hat{f}(x_0)](f(x_0) - f(x_0)) + \text{Var}[Y_0] \\ &\quad (\text{Variance definition and } E[Y_0] = f(x_0)) \\ &= \text{Var}[\hat{f}(x_0)] + (\text{Bias}[\hat{f}(x_0)])^2 + 0 + \sigma^2 \quad (\text{Var}[Y_0] = \sigma^2) \end{aligned}$$

$$E[(\hat{Y}_0 - Y_0)]^2 = (\text{Bias}[\hat{f}(x_0)])^2 + \text{Var}[\hat{f}(x_0)] + \sigma^2$$

The U-Shaped Test MSE: A Bias-Variance Tradeoff

- **Left side of the U (Underfitting):** High bias, low variance, and low flexibility. The model is too simple, resulting in poor predictions and a high test MSE.
- **Bottom of the U (Optimal Fit):** Low bias, low variance. The model strikes the ideal balance, achieving minimal test MSE and good generalization.
- **Right side of the U (Overfitting):** Low bias, high variance, and high flexibility. The model is too complex, fitting noise in the training data, leading to poor performance on new data and a rising test MSE.

R Code

#input datasets

```
trainData <- read.csv("1-training_data(1).csv")
```

```
testData <- read.csv("1-test_data(1).csv")
```

#separate x columns

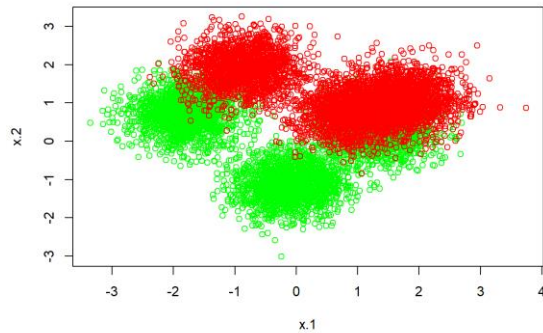
```
trainDataX <- cbind(trainData$x.1, trainData$x.2)
```

```
testDataX <- cbind(testData$x.1, testData$x.2)
```

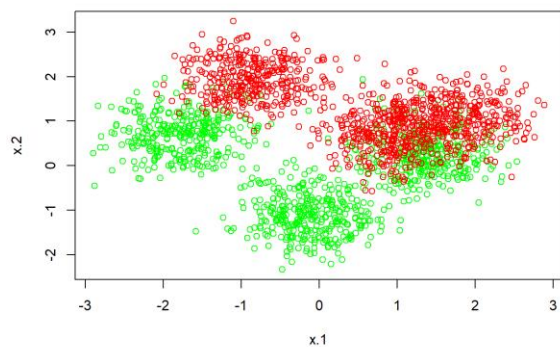
```
library(class)      #used for knn()
```

#scatter plots as exploratory data analysis

```
plot(trainDataX, xlab = "x.1", ylab = "x.2", col = ifelse(trainData$y ==  
  "yes", "green", "red"))
```



```
plot(testDataX, xlab = "x.1", ylab = "x.2", col = ifelse(testData$y ==  
  "yes", "green", "red"))
```



Question 1

#(b)

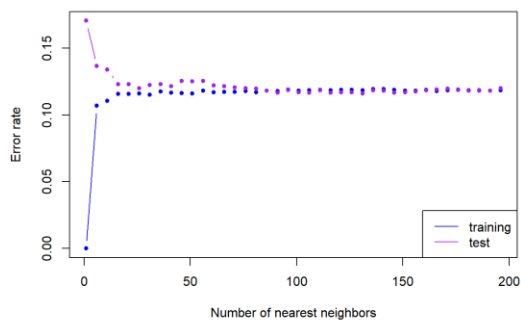
#fit knn with k

```
k <- seq(1, 200, by = 5)
nk <- length(k)
err.rate.train <- numeric(length = nk)
err.rate.test <- numeric(length = nk)
names(err.rate.train) <- names(err.rate.test) <- k
```

```
for (i in seq_along(k))
{
  set.seed(1)
  mod.train <- knn(trainDataX, trainDataX, trainData$y, k = k[i])
  set.seed(1)
  mod.test <- knn(trainDataX, testDataX, trainData$y, k = k[i])
  err.rate.train[i] <- 1 - sum(mod.train == trainData$y) / length(trainData$y)
  err.rate.test[i] <- 1 - sum(mod.test == testData$y) / length(testData$y)
}
```

#plot error rates

```
plot(k, err.rate.train,
     xlab = "Number of nearest neighbors", ylab = "Error rate",
     type = "b", ylim = range(c(err.rate.train, err.rate.test)), col = "blue", pch = 20)
lines(k, err.rate.test, type = "b", col = "purple", pch = 20)
legend("bottomright", lty = 1, col = c("blue", "purple"), legend = c("training", "test"))
```



```
result <- data.frame(k, err.rate.train, err.rate.test)
```

#(c)

#the row with the minimum test error

```
optimal_k_row <- which.min(result$err.rate.test)
```

```
#its corresponding 'k' value
k.opt <- result$k[optimal_k_row]
```

```
print(paste("Optimal k:", k.opt))
```

```
## [1] "Optimal k: 131"
result[result$k == k.opt, ]
```

	k <dbl>	err.rate.train <dbl>	err.rate.test <dbl>
131	131	0.1185	0.116

1 row

```
#search for where the difference between the error rates is smallest
rateDiff <- which.min(result$err.rate.test - result$err.rate.train)
```

```
result$k[rateDiff]
## [1] 131
```

```
#in both cases, I end up with k = 131
```

#(d)

```
n.grid <- 40
x1.grid <- seq(f = min(trainDataX[, 1]), t = max(trainDataX[, 1]), l = n.grid)
x2.grid <- seq(f = min(trainDataX[, 2]), t = max(trainDataX[, 2]), l = n.grid)
grid <- expand.grid(x1.grid, x2.grid)
```

```
set.seed(1)
mod.opt <- knn(trainDataX, grid, trainData$y, k = k.opt, prob = T)
prob <- attr(mod.opt, "prob") #prob is voting fraction for winning class
prob <- ifelse(mod.opt == "yes", prob, 1 - prob) #now it is voting fraction for y == "yes"
prob <- matrix(prob, n.grid, n.grid)
```

```
plot(trainDataX, col = ifelse(trainData$y == "yes", "green", "red"))
```

```
#decision boundary
contour(x1.grid, x2.grid, prob,
  levels = 0.5, labels = "", xlab = "", ylab = "",
  main = "", add = T)
```